

Table of Content

1	WEB SCRAPING: A KEY TOOL IN DATA SCIENCE	4
1.1	INTRODUCTION	4
1.2	IMPORTANCE OF WEB SCRAPING IN DATA SCIENCE	4
1.2.1	DATA COLLECTION	4
1.2.2	REAL-TIME APPLICATIONS	4
1.2.3	MACHINE LEARNING	4
1.3	WEB SCRAPING WITH PYTHON	4
1.3.1	BEAUTIFULSOUP	4
1.3.2	SCRAPY	4
1.3.3	SELENIUM	5
1.4	APPLICATIONS OF WEB SCRAPING	5
1.4.1	PRICE COMPARISON	5
1.4.2	EMAIL ADDRESS GATHERING	5
1.4.3	SOCIAL MEDIA SCRAPING	5
1.5	CONCLUSION	5
2	INTRODUCTION TO HTML FOR WEB SCRAPING	6
2.1	OVERVIEW	6
2.2	UNDERSTANDING HTML STRUCTURE	6
2.2.1	BASIC HTML ELEMENTS	6
2.3	COMPOSITION OF AN HTML TAG	6
2.3.1	EXAMPLE: THE ANCHOR TAG	6
2.3.2	COMPONENTS OF A TAG	6
2.4	HTML AS A DOCUMENT TREE	7
2.4.1	PARENT-CHILD RELATIONSHIPS	7
2.5	UNDERSTANDING HTML TABLES	7
2.5.1	EXAMPLE	7
2.6	SUMMARY	7
3	INTRODUCTION TO WEB SCRAPING	8
3.1	OVERVIEW	8
3.1.1	WHY USE WEB SCRAPING?	8
3.2	WHAT IS WEB SCRAPING?	8
3.3	UNDERSTANDING BEAUTIFULSOUP	8
3.3.1	CREATING A BEAUTIFULSOUP OBJECT	8
3.3.2	HTML TREE NAVIGATION	9
3.3.3	EXAMPLE – ACCESSING HTML ELEMENTS	9

3.3.4	ACCESSING ATTRIBUTES AND TEXT	9
3.4	USING THE FIND_ALL() METHOD	9
3.4.1	EXAMPLE – EXTRACTING TABLE DATA	9
3.5	APPLYING BEAUTIFULSOUP TO A WEB PAGE	9
3.5.1	STEPS TO SCRAPE A WEBPAGE	10
3.6	SUMMARY	10
4	PROJECT OVERVIEW	10
4.1	INTRODUCTION	10
4.2	DASHBOARD ANALYTICS DISPLAYED	10
4.2.1	LEARNING OBJECTIVES	11
4.3	TOOLS AND PLATFORMS	11
4.3.1	JUPYTER NOTEBOOK	11
4.3.2	IBM WATSON STUDIO	11
4.4	REVIEW CRITERIA	11
4.4.1	EVALUATION COMPONENTS	11
5	UNDERSTANDING STOCK AND SHARES	12
5.1	WHAT IS A STOCK?	12
5.1.1	KEY POINTS	12
5.2	BUYING AND SELLING STOCKS	12
5.2.1	HOW INVESTORS PROFIT OR LOSE	12
5.2.2	FACTORS INFLUENCING STOCK PRICE	12
5.3	STOCK TRADING AND TICKER	12
5.3.1	USING YFINANCE API	12
6	THE YFINANCE PYTHON LIBRARY	13
6.1	INTRODUCTION	13
6.2	EXAMPLE: DOWNLOADING HISTORICAL STOCK PRICES	13
6.3	EXPLANATION OF THE CODE	13
6.3.1	STEP 1: IMPORT THE YFINANCE LIBRARY	13
6.3.2	STEP 2: CREATE A TICKER OBJECT	13
6.3.3	STEP 3: DOWNLOAD HISTORICAL DATA	13
6.3.4	STEP 4: DISPLAY THE DOWNLOADED DATA	14
7	GAMESTOP STOCK VS TESLA	14
7.1	INTRODUCTION	14
7.2	HOW SHORT SELLING WORKS	14
7.3	CASE STUDY: TESLA — WHEN SHORT SELLERS WERE WRONG	14

7.4	CASE STUDY: GAMESTOP — A RETAIL-DRIVEN SURGE	14
7.5	TASK:	15
8	NEXT STEPS	15
8.1	RECOMMENDED NEXT COURSES	15
8.1.1	1. DATABASES AND SQL FOR DATA SCIENCE WITH PYTHON	15
8.1.2	2. DATA ANALYSIS WITH PYTHON	15
8.1.3	3. DATA VISUALIZATION WITH PYTHON	15
8.2	PROFESSIONAL LEARNING PATHS	15
8.2.1	IBM DATA ANALYTICS PROFESSIONAL CERTIFICATE	15
8.2.2	IBM DATA SCIENCE PROFESSIONAL CERTIFICATE	15

1 Web Scraping: A Key Tool in Data Science

1.1 Introduction

Web scraping, also known as web harvesting or web data extraction, is a technique used to extract large amounts of data from websites. Since data on websites is usually unstructured, web scraping helps convert it into a structured format suitable for analysis.

1.2 Importance of Web Scraping in Data Science

Web scraping plays an integral role in the field of data science. It is used for various purposes such as:

1.2.1 Data Collection

Web scraping is a primary method of collecting data from the internet. The collected data can then be used for analysis, research, and other purposes.

1.2.2 Real-time Applications

It is used in real-time scenarios like weather updates, stock tracking, and price comparison.

1.2.3 Machine Learning

Web scraping provides essential datasets used for training machine learning models.

1.3 Web Scraping with Python

Python offers several libraries that simplify the process of web scraping. Some of the most popular ones are:

1.3.1 BeautifulSoup

BeautifulSoup is a Python library used to extract data from HTML and XML files. It creates a parse tree from the page source code, making data extraction easy and readable.

```
from bs4 import BeautifulSoup
import requests
```

```
URL = "http://www.example.com"
page = requests.get(URL)
soup = BeautifulSoup(page.content, "html.parser")
```

1.3.2 Scrapy

Scrapy is an open-source web crawling framework for Python used to extract data from websites.

```
import scrapy
```

```
class QuotesSpider(scrapy.Spider):
    name = "quotes"
    start_urls = ['http://quotes.toscrape.com/tag/humor/']

    def parse(self, response):
        for quote in response.css('div.quote'):
            yield {'quote': quote.css('span.text::text').get()}
```

1.3.3 Selenium

Selenium is a tool used for controlling web browsers programmatically and automating browser tasks.

```
from selenium import webdriver
```

```
driver = webdriver.Firefox()
driver.get("http://www.example.com")
```

1.4 Applications of Web Scraping

Web scraping is widely used across multiple fields for different applications:

1.4.1 Price Comparison

Services like **ParseHub** use web scraping to collect data from online stores and compare product prices.

1.4.2 Email Address Gathering

Companies use web scraping to collect email addresses for marketing and bulk emailing purposes.

1.4.3 Social Media Scraping

Web scraping helps collect data from platforms like **Twitter** to analyze trends, sentiments, and public interests.

1.5 Conclusion

Web scraping is an essential skill in the fast-growing field of data science. It enables data professionals to transform the web into a vast source of structured, analyzable data. However, it is crucial to use web scraping **ethically and responsibly**, respecting each website's **terms of use** and **robots.txt** policies.

2 Introduction to HTML for Web Scrapping

2.1 Overview

We will review **Hypertext Markup Language (HTML)** and its importance in **web scraping**. Many websites contain valuable data, such as real estate listings, stock prices, or coding solutions. Understanding HTML allows you to extract this data efficiently using Python.

By the end of this lesson, you will:

- Review the structure of a basic HTML page.
- Understand the composition of an HTML tag.
- Explore HTML trees and relationships between elements.
- Learn how HTML tables organize data.

2.2 Understanding HTML Structure

A web page is primarily built using HTML. HTML documents contain **tags** enclosed in angle brackets (e.g., `<tag>`). These tags define how web content appears in a browser.

2.2.1 Basic HTML Elements

- `<!DOCTYPE html>`: Declares that the document is an HTML file.
- `<html>`: The root element of the HTML page.
- `<head>`: Contains metadata and other non-visual elements.
- `<body>`: Contains all visible content on the page.

For instance, player names might appear inside `<h3>` tags (headings), while salaries may appear within `<p>` tags (paragraphs). The combination of these tags structures the content you see on a web page.

2.3 Composition of an HTML Tag

HTML elements consist of several key components:

2.3.1 Example: The Anchor Tag

```
<a href="https://www.ibm.com">IBM</a>
```

This tag creates a hyperlink labeled **IBM**, which redirects users to **IBM.com** when clicked.

2.3.2 Components of a Tag

- **Tag Name**: Defines the type of element (e.g., `a` for link, `p` for paragraph).
- **Opening and Closing Tags**: Most elements start with `<tag>` and end with `</tag>`.
- **Content**: The text or element displayed on the webpage.
- **Attributes**: Provide additional information about elements, defined as name–value pairs (e.g., `href="https://www.ibm.com"`).

Think of HTML tags as Python classes, where each individual tag represents an instance.

You can inspect a web page's structure using the **Inspect Element** feature in a browser. This helps visualize the HTML elements and their relationships. Real web pages often include **CSS** and **JavaScript**, which are not covered here.

2.4 HTML as a Document Tree

Each HTML document can be visualized as a **tree structure** where elements are nested within one another.

2.4.1 Parent-Child Relationships

- The `<html>` tag contains `<head>` and `<body>` tags.
- `<head>` and `<body>` are **children** of `<html>`.
- `<title>` is a **child** of `<head>` and a **descendant** of `<html>`.
- `<h3>` and `<p>` are **children** of `<body>` and **siblings** of each other.
- `<b>` (bold) can be a **child** of `<h3>`.

Each nested level represents a deeper layer in the HTML document tree, similar to nodes in a family tree.

2.5 Understanding HTML Tables

HTML tables organize data in a structured format. They are defined using a set of nested tags:

- `<table>`: Defines the table structure.
- `<tr>`: Defines a table row.
- `<th>`: Defines a header cell (usually the first row).
- `<td>`: Defines a data cell.

2.5.1 Example

```
<table>
<tr>
  <th>Player Name</th>
  <th>Salary</th>
</tr>
<tr>
  <td>John Doe</td>
  <td>$1,000,000</td>
</tr>
</table>
```

This example shows how to represent player names and salaries in a structured table format.

2.6 Summary

- HTML defines the structure and layout of web pages using tags.

- Understanding HTML is essential for **web scraping** since most online data resides within HTML elements.
- Key HTML concepts include **tags**, **attributes**, **document trees**, and **tables**.
- Using Python libraries like **BeautifulSoup**, you can extract and analyze data from these HTML structures.

3 Introduction to Web Scraping

3.1 Overview

Now, we will cover **Web Scraping**. By the end of this lesson, you will be able to:

- Define web scraping.
- Understand the role of **BeautifulSoup objects**.
- Apply the **find_all()** method.
- Perform web scraping on a website using Python.

3.1.1 Why Use Web Scraping?

Imagine you want to analyze hundreds of data points to find the best players in a sports team. Manually copying and pasting information from different websites into a spreadsheet would be time-consuming and overwhelming. **Web scraping** solves this problem by automatically extracting information from websites within minutes using Python.

3.2 What is Web Scraping?

Web scraping is the process of **automatically extracting data** from a website. It allows you to collect large amounts of data quickly and efficiently.

To get started, we need two Python libraries:

- **Requests**: Used to download a web page.
- **BeautifulSoup**: Used to parse and navigate the HTML content.

3.3 Understanding BeautifulSoup

BeautifulSoup is a Python library used to parse HTML and XML documents. It represents the structure of a web page as a nested data structure (a tree of objects), allowing easy navigation and data extraction.

3.3.1 Creating a BeautifulSoup Object

To parse a document, pass the HTML content into the **BeautifulSoup()** constructor:

```
from bs4 import BeautifulSoup
soup = BeautifulSoup(HTML, 'html.parser')
```

The soup object represents the entire HTML document as a **tree structure**.

3.3.2 HTML Tree Navigation

Each HTML element is represented as a **tag object** in BeautifulSoup.

- The tag `<title>` corresponds to the title of the document.
- If multiple tags have the same name, BeautifulSoup selects the **first occurrence** by default.
- You can access a tag's **child elements** or **navigate up the tree** using the parent attribute.
- The `next_sibling` attribute allows you to move to the next tag at the same level.

3.3.3 Example – Accessing HTML Elements

```
tag = soup.h3 # Access the first <h3> tag
child = tag.b # Access the <b> tag inside <h3>
parent = tag.parent # Access the parent of <h3>
```

3.3.4 Accessing Attributes and Text

Attributes of a tag can be accessed as **key-value pairs** in a dictionary:

```
tag['class'] # Returns the class attribute value
```

To extract text, use the **NavigableString** object:

```
tag.string # Returns the text inside a tag
```

3.4 Using the `find_all()` Method

The `find_all()` method filters and retrieves elements that match certain criteria, such as tag name, attribute, or text.

3.4.1 Example – Extracting Table Data

Suppose you have a table of pizza places:

```
table = BeautifulSoup(HTML, 'html.parser')
rows = table.find_all('tr')
```

Each `<tr>` element corresponds to a table row. You can then iterate through the rows and cells:

```
for row in rows:
    cells = row.find_all('td')
    for cell in cells:
        print(cell.text)
```

This allows you to extract structured data from HTML tables easily.

3.5 Applying BeautifulSoup to a Web Page

To scrape data from a live webpage, use the **Requests** library to download it and **BeautifulSoup** to parse it.

3.5.1 Steps to Scrape a Webpage

1. Import the Required Modules:

```
import requests
from bs4 import BeautifulSoup
```

2. Send a GET Request to download the page:

```
url = 'https://example.com'
page = requests.get(url)
```

3. Access the HTML content using the text attribute:

```
html_content = page.text
```

4. Create a BeautifulSoup object to parse the HTML:

```
soup = BeautifulSoup(html_content, 'html.parser')
```

Now, you can use methods like `find_all()` to locate and extract the desired data.

3.6 Summary

- **Web scraping** automates the extraction of data from websites.
- **Requests** helps download web pages, while **BeautifulSoup** parses HTML.
- The **BeautifulSoup object** represents a document as a tree structure.
- Methods like **find_all()** help filter and extract specific tags or text.
- Web scraping makes it easy to collect and analyze large amounts of data efficiently.

4 Project Overview

4.1 Introduction

For this project, you will take on the role of a **Data Scientist / Data Analyst** working for a new startup investment firm that assists customers in investing their money in stocks. Your primary responsibilities include:

- Extracting financial data such as historical share prices and quarterly revenue reports from various sources using Python libraries and web scraping.
- Visualizing the collected data in a dashboard to identify patterns and trends.

The stocks to be analyzed in this project are **Tesla**, **Amazon**, **AMD**, and **GameStop**.

4.2 Dashboard Analytics Displayed

A **dashboard** provides a visual summary of key performance indicators (KPIs) in a clear and concise manner. In this project, you will practice analyzing data sets and extracting KPIs for visualization.

4.2.1 Learning Objectives

- Access and display data in dashboards using Python.
- Identify and visualize key performance indicators.
- Gain practical experience in using **Plotly** for data visualization.

Note: Plotly will be used for visualization in this course, but it is not a prerequisite to take this course.

4.3 Tools and Platforms

- Jupyter Notebook
- Watson Studio

4.3.1 Jupyter Notebook

Used for AI and Data Science-related projects.

4.3.2 IBM Watson Studio

A component of **IBM Cloud Pak for Data**, Watson Studio is a suite of tools and a collaborative environment for:

- Data Scientists
- Data Analysts
- AI and Machine Learning Engineers
- Domain Experts

These professionals use Watson Studio to **develop**, **analyze**, and **deploy** data projects collaboratively.

4.4 Review Criteria

The project consists of two hands-on labs on **Extracting Stock Data** and one **assignment** to complete.

4.4.1 Evaluation Components

- **Two Quizzes:** Based on the output of your hands-on labs.
- **Peer Review Assignment:** You will share screenshots and outcomes from your completed assignment for peer evaluation.

By completing these tasks, you will demonstrate your ability to extract, analyze, and visualize financial data effectively using Python.

5 Understanding Stock and Shares

5.1 What is a Stock?

A **stock**, also known as **equity**, is a type of security that represents ownership of a fraction of a **corporation**. Owning stock entitles an investor to a proportion of the corporation's **assets** and **profits** based on the number of shares they hold.

5.1.1 Key Points

- A stock share is essentially a piece of ownership in a company.
- Units of stock are called **shares**.
- Shareholders may receive a portion of the company's profits in the form of **dividends**.

5.2 Buying and Selling Stocks

An **investor** can buy stocks and later sell them to earn a profit or incur a loss, depending on market performance.

5.2.1 How Investors Profit or Lose

- If the stock price **increases**, the investor makes a **profit** when selling.
- If the stock price **decreases**, the investor incurs a **loss**.

5.2.2 Factors Influencing Stock Price

Determining the stock price is a complex process and depends on several factors, such as:

- The number of **outstanding shares**.
- The company's **future profit potential**.
- **Market conditions** and **investor sentiment**.

5.3 Stock Trading and Ticker

People trade stocks throughout the day on various stock exchanges. The **stock ticker** continuously reports the current price of specific stocks during trading sessions.

5.3.1 Using yFinance API

In this practice, you will:

- Use the **y-finance API** to obtain stock ticker information.
- Extract detailed data about a company's stock.
- Analyze and interpret results through data exploration and visualization.

6 The yfinance Python Library

6.1 Introduction

The **yfinance** is a Python library with a user-friendly interface for downloading historical market data from **Yahoo Finance**. It allows users to obtain historical stock prices, dividends, and other financial data for **stocks**, **exchange-traded funds (ETFs)**, and other securities.

6.2 Example: Downloading Historical Stock Prices

Below is an example of using **yfinance** to download historical stock prices.

```
import yfinance as yf

# Download historical data for a stock
msft = yf.Ticker("MSFT")
msft_data = msft.history(period="max")

# Display the downloaded data
msft_data.head()
```

6.3 Explanation of the Code

6.3.1 Step 1: Import the yfinance Library

Import the **yfinance** library using the alias **yf**.

6.3.2 Step 2: Create a Ticker Object

Create a **Ticker** object for the Microsoft stock using:

```
yf.Ticker("MSFT")
```

This object represents the Microsoft stock symbol (**MSFT**).

6.3.3 Step 3: Download Historical Data

Use the **history()** method of the Ticker object to download historical data for the stock. The **period** parameter specifies the time frame of the data to download.

6.3.3.1 Common Values for the period Parameter

- **period="1d"**: 1 day of historical data
- **period="5d"**: 5 days of historical data
- **period="1mo"**: 1 month of historical data
- **period="3mo"**: 3 months of historical data
- **period="6mo"**: 6 months of historical data
- **period="1y"**: 1 year of historical data
- **period="2y"**: 2 years of historical data
- **period="5y"**: 5 years of historical data
- **period="10y"**: 10 years of historical data
- **period="ytd"**: Data since the beginning of the current year

- `period="max"`: All available historical data

6.3.4 Step 4: Display the Downloaded Data

Use the **head()** function to print the first few rows of the downloaded data:

```
msft_data.head()
```

This displays a **Pandas DataFrame** containing Microsoft's historical stock prices and other financial data.

7 Gamestop Stock vs Tesla

7.1 Introduction

Determining a stock's price is complex — it depends on many factors such as the number of outstanding shares, expected future profits, and investor sentiment. A key driver is a company's profit growth: if profits are expected to increase, the stock price will often rise. If you expect the price to fall, investors can use *short selling* to profit from declines.

7.2 How Short Selling Works

- **Mechanics:** An investor borrows shares, sells them on the market, then (later) buys shares back to return to the lender.
- **Profit if price falls:** If the repurchase price is lower than the sale price, the short seller keeps the difference.
- **Risk:** If the stock price rises, losses are potentially unlimited because the short seller must buy back at a higher price.
- **Typical participants:** Short selling is usually performed by experienced investors (e.g., hedge funds).
- **Behavior:** Stocks can fall faster than they rise, so short positions sometimes realize gains more quickly — but they are riskier if sentiment reverses.

7.3 Case Study: Tesla — When Short Sellers Were Wrong

- Many investors heavily shorted Tesla when the company was unprofitable or growth was uncertain.
- As Tesla became consistently profitable and showed increasing profits, its stock price rose strongly.
- Short sellers who persisted faced large losses because the price movement was driven by improving fundamentals rather than temporary market noise.

7.4 Case Study: GameStop — A Retail-Driven Surge

- GameStop, a brick-and-mortar video-game retailer struggling financially, was heavily shorted by hedge funds.
- Users of the Reddit forum *WallStreetBets* coordinated large-scale buying of GameStop shares.

- The sudden surge in demand caused GameStop's share price to soar, producing substantial losses for hedge funds with short positions.
- This rally was driven largely by retail sentiment and momentum rather than company fundamentals, making it less sustainable.

7.5 Task:

- As a data scientist working for a hedge fund, you will extract the profit data for Tesla and GameStop and build a dashboard to compare the price of the stock vs the profit for the hedge fund.

8 Next Steps

8.1 Recommended Next Courses

As a next step, you can explore the following courses to further enhance your skills in **Data** and **Python**:

8.1.1 1. Databases and SQL for Data Science with Python

Learn how to work with relational databases, write SQL queries, and integrate SQL with Python for data-driven applications.

8.1.2 2. Data Analysis with Python

Develop your ability to clean, analyze, and interpret data using Python's powerful data analysis libraries such as **Pandas** and **NumPy**.

8.1.3 3. Data Visualization with Python

Master the art of visual storytelling with Python's visualization tools like **Matplotlib**, **Seaborn**, and **Plotly**.

8.2 Professional Learning Paths

If you aim to start a career in **Data Analytics** or **Data Science**, these courses are part of the following professional certificate programs:

8.2.1 IBM Data Analytics Professional Certificate

Designed to equip you with essential skills in data collection, analysis, and visualization to become job-ready in the field of Data Analytics.

8.2.2 IBM Data Science Professional Certificate

A comprehensive program covering Python, data analysis, machine learning, and other essential tools to prepare you for a successful career in Data Science.